SUPER DEV BUILDERS · DOKUMENTASI PENGGUNA

Paket Update Menyeluruh — Super Dev Builders

Semua detail SDB: 16 tahap, 12 divisi, gate, perkakas, bot Telegram, bukti, dan batasan — bahasa awam

dokumentasi | tanpa dependensi runtime

Versi dokumen: 1.0 · **Tanggal:** 4 November 2026 · **Disusun oleh:** Divisi Ekosistem SDB **Untuk:** Mas Anas (pemilik) · **Sifat:** penjelasan lengkap, bahasa awam, semua angka terverifikasi

> **Cara membaca dokumen ini.** Kalau Anda hanya punya 3 menit, baca **Bagian 1** saja. Kalau ingin tahu "apa saja isinya", baca **Bagian 4–7**. Kalau ingin langsung memakai, lompat ke **Bagian 13 (cheat sheet)**. Semua istilah asing dijelaskan di **Bagian 17 (glosarium)**.

Daftar Isi

1. Ringkasan tiga menit
2. Apa itu Super Dev Builders — dengan analogi
3. Masalah yang dipecahkan SDB
4. Prinsip #0: keamanan lebih dulu, bukan kecepatan
5. Alur kerja 16 tahap — dari ide sampai serah terima
6. Dua belas divisi: siapa mengerjakan apa
7. Gerbang (gate) dan bukti: kenapa "sudah selesai" tidak cukup
8. Isi rumah: struktur folder dan artinya
9. Perkakas: 31 skrip, 14 perintah `sdb`, 470 tes
10. Skill dan tool: 40 skill sendiri, 17 tool terpasang
11. Ruang kendali (Command Deck) dan control plane
12. Bot Telegram SDB: tiga kanal operasional
13. Alur membuat produk lewat Telegram — langkah demi langkah
14. Keamanan berlapis: apa yang dijaga dan bagaimana
15. Cara pakai harian — cheat sheet
16. Batasan jujur: apa yang belum ada

17. Glosarium bahasa awam
18. Peta dokumen rujukan
19. Lampiran A — angka terverifikasi dan cara mengeceknya
20. Lampiran B — riwayat dokumen

---

1. Ringkasan tiga menit

SDB adalah "pabrik perangkat lunak" pribadi yang bekerja dengan prosedur tetap. Ia bukan satu program ajaib, melainkan kumpulan aturan + alat + agen AI yang dijalankan dengan urutan yang sama setiap kali: **izin dulu → pahami → rencanakan → bangun → periksa → bukti → serahkan.**

Tiga hal yang membuatnya berbeda dari "minta AI bikin aplikasi":

Hal	Tanpa SDB	Dengan SDB
Izin	Langsung kerjakan	**Gate #0**: harus ada dasar otorisasi tertulis, kalau tidak → berhenti
Bukti	"Sudah jadi, percaya saja"	Perintah + output nyata ditempel di `VERIFY.md`; gate menolak PASS tanpa artefak
Keamanan	Terserah model	4 kelas risiko, isolasi proses, pemindaian rahasia, approval manusia untuk aksi berisiko

Kondisinya per 4 November 2026 (semua angka bisa Anda cek sendiri, lihat Lampiran A):

- 130 commit, 103 berkas Python, 18.748 baris kode, 36 berkas tes, **470 tes**.
- **40 skill buatan sendiri** + 17 tool terpasang (versi dipin) + 8 paket divisi.
- **12 divisi (D0–D11)**, **16 tahap kerja**, **4 kelas risiko**.
- 3 layanan aktif: bot Telegram, pekerja build, dan Command Deck; 3 timer: laporan 4 jam,
- pemantau CI, dan pemeriksaan kesehatan deck.
- Bot Telegram sudah bisa **melapor otomatis tiap 4 jam**, **menjawab pertanyaan**, dan
- **menerima perintah membuat produk** dengan alur dua tahap (spesifikasi → persetujuan → build).

Satu kalimat: SDB mengubah "minta AI menulis kode" menjadi "proyek kecil yang punya izin, rencana, bukti uji, dan jejak audit".

---

2. Apa itu Super Dev Builders — dengan analogi

2.1 Analogi utama: bengkel mobil dengan SOP

Bayangkan bengkel yang menerima mobil pelanggan. Bengkel biasa: mekanik langsung buka kap mesin. Bengkel bersertifikat:

1. **Cek surat-surat dulu** — ini mobil siapa, boleh dibongkar atau tidak. (Gate #0)
2. **Diagnosa** — keluhan apa, gejala apa, dicatat. (Tahap 01–03)
3. **Rancang perbaikan** — gambar kerja, part apa. (Tahap 04–06)
4. **Kerjakan** — mekanik berbeda untuk mesin, kelistrikan, bodi. (Tahap 07, divisi D4/D5)
5. **Periksa mutu** — mekanik lain memeriksa pekerjaan mekanik pertama. (Tahap 08–09, D6/D7)
6. **Uji keselamatan** — rem, lampu, emisi. (Tahap 10–11, D8/D9)
7. **Serah terima dengan berita acara** — bukan "sudah pak, silakan". (Tahap 12–16)

SDB melakukan hal yang sama untuk **perangkat lunak**: website, aplikasi, program, skrip, bahkan membangun ulang aplikasi yang Anda miliki izinnya.

2.2 Analogi kedua: dapur restoran

- **Resep** = workflow 16 tahap (tidak berubah tiap hari).
- **Koki** = agen AI (model berbeda untuk tugas berbeda — yang murah untuk tugas rutin, yang mahal untuk arsitektur dan keamanan).
- **Bahan** = artefak: requirement, ADR (catatan keputusan), diff kode, hasil tes, laporan scan.
- **Juru rasa** = gate deterministik yang menolak hidangan kalau kurang asin — dan juru rasa ini **bukan AI**, jadi tidak bisa dibujuk.

2.3 Tiga kata yang perlu dipegang

Kata	Arti praktis
Otonom	Bisa jalan sendiri (laporan 4 jam, build bertahap) tanpa Anda menunggui
Terkendali	Setiap langkah berisiko punya gerbang: izin, approval, rollback
Terbukti	Klaim hanya sah kalau ada artefak yang bisa diperiksa mesin, bukan kalimat

3. Masalah yang dipecahkan SDB

SDB tidak dibuat untuk "seru-seruan bikin website". Ia menjawab lima masalah nyata:

1. **AI suka mengarang status.** "Sudah saya perbaiki" padahal belum. SDB memaksa output berupa artefak: log tes, log build, verdict JSON. Kalau tidak ada → FAIL, bukan PASS.
2. **AI suka melebar.** Satu permintaan kecil jadi refactor seluruh repo. SDB membatasi dengan spesifikasi + rencana tugas kecil (1 tugas = 1 berkas + 1 cara verifikasi).
3. **Pekerjaan berisiko diperlakukan sama dengan pekerjaan ringan.** SDB membagi risiko LOW/MEDIUM/HIGH/CRITICAL. Aksi destruktif produksi **tidak boleh otonom**.
4. **Kontek hilang antar sesi.** SDB menyimpan keputusan sebagai dokumen (ADR, blueprint, `memory`, `VERIFY.md`), bukan mengandalkan ingatan percakapan.
5. **Pemilik tidak punya jendela pandang.** Command Deck + laporan Telegram 4 jam memberi visibilitas tanpa harus membuka terminal.

---

4. Prinsip #0: keamanan lebih dulu, bukan kecepatan

Urutan prioritas SDB, tidak bisa dibalik:

```
1. Keamanan & privasi (termasuk PC ini, kredensial, dan data)
2. Kebenaran (bukti, bukan klaim)
3. Kecepatan menuju hasil
```

Konsekuensi yang terlihat sehari-hari:

- **Gate #0 wajib** sebelum menyentuh target yang bukan milik sendiri — termasuk membangun ulang aplikasi. Tanpa dasar otorisasi → proses berhenti, bukan "lanjut pelan-pelan".
- **Rahasia tidak masuk prompt/log.** Redaksi (`scripts/redact_network.py`, `secret_scan`) dijalankan sebelum artefak disimpan.
- **Token bot tidak pernah diwariskan** ke proses agen yang sedang membangun (dihapus dari environment anak), sehingga kode hasil build tidak bisa "membaca" token tersebut.
- **Aksi HIGH/CRITICAL berhenti minta manusia.** Termasuk deploy produksi, migrasi DB, rotasi kredensial.

---

5. Alur kerja 16 tahap — dari ide sampai serah terima

Ini "resep" yang sama untuk semua proyek SDB. Dijalankan oleh `scripts/workflow_run.py` (dengan artefak per tahap), dan gate-nya oleh `scripts/gate_check.py` + `scripts/release_check.py`.

#	Tahap	Bahasa awam	Siapa
01	User objective	Anda menyebut keinginan	Anda + D0
01A	**Authorization / Scope Gate**	Cek surat izin. Gagal = stop	D0 + `gate_check.py`
—	RE & reconstruction	Kalau target sudah ada: bedah dulu, pisahkan FAKTA / DUGAAN / TIDAK TAHU	D1
02	Orchestrator clarification	Tanya yang belum jelas, **dilarang menebak**	D0
03	Research	Riset seperlunya, output pendek	D2
04	Architecture + ADR	Rancang bentuk + catat alasan keputusan	D3
05	Human design approval	Anda menyetujui desain (proyek besar)	Anda
06	Implementation plan	Pecah jadi tugas kecil + cara verifikasinya	D0/D3
07	Frontend / backend	Menulis kode dengan TDD	D4/D5
08	Code review	Diperiksa orang (agen) lain	D6
09	QA / integration / E2E	Diuji, bukan diasumsikan	D7
10	Security	SAST, dependency, secret, config scan	D8
11	Code audit & assurance	Cek "apakah ini masih sesuai rancangan"	D9
12	**Release gate**	Berkas bukti diperiksa mesin; kedaluwarsa 24 jam	D10 + `release_check.py`

#	Tahap	Bahasa awam	Siapa
13	Human approval	Anda menyetujui rilis produksi	Anda
14	DevOps deploy	Pasang, satu layanan sekaligus	D10
15	Observability / smoke / rollback	Pantau, uji asap, siapkan jalan pulang	D10
16	Documentation + handover	README, runbook, catatan rilis	D11

Kenapa 16 tahap, bukan 3? Karena kesalahan termahal terjadi di sambungan antar tahap, bukan di dalam tahap. Contoh: kode bagus tapi tidak ada rollback → insiden jadi panjang.

---

6. Dua belas divisi: siapa mengerjakan apa

Divisi bukan gedung, melainkan **peran**. Satu agen bisa memegang beberapa peran, tetapi tanggung jawabnya dipisah supaya ada pemeriksa independen.

Divisi	Nama	Tugas inti	Agen
D0	Master Orchestrator / Product & Planning	Menerima permintaan, menjalankan Gate #0, memecah pekerjaan	`sdb-orchestrator`
D1	Reverse Engineering & Reconstruction	Bedah target berizin, kumpulkan bukti, ukur kemiripan	`sdb-recon`
D2	Research	Riset teknologi, perbandingan, sumber	`sdb-orchestrator`, `sdb-architect`
D3	Architecture	Rancangan sistem, ADR, pemilihan stack	`sdb-architect`
D4/D5	Engineering Frontend & Backend	Menulis kode	`sdb-frontend`, `sdb-builder`
D6	Code Review	Menilai kode orang lain	`sdb-reviewer`
D7	QA / Test	Uji unit, integrasi, E2E	`sdb-reviewer`

Divisi	Nama	Tugas inti	Agen
D8	Security	Scan & triase keamanan	`sdb-security`
D9	Code Audit & Engineering Assurance	Audit menyeluruh, **independen** dari pembuat	`sdb-security`
D10	DevOps / Release / Reliability	Deploy, observabilitas, rollback	`sdb-devops`
D11	Documentation	README, runbook, handover	`sdb-devops`

Aturan pemisahan: pembuat tidak boleh menjadi satu-satunya pemeriksa. D6/D9 sengaja berbeda peran dari D4/D5 supaya "sudah saya cek sendiri" tidak berlaku.

---

7. Gerbang (gate) dan bukti: kenapa "sudah selesai" tidak cukup

7.1 Gate #0 — pintu izin

Berkas `evidence/<target>/authorization.yaml` harus memuat, antara lain:

- nama target (slug valid),
- jenis otorisasi: `owned` (milik sendiri), `open_source`, `staging`, atau `written_permission`,
- daftar target yang diizinkan (URL / repo / app / binary),
- batas kedalaman analisis untuk tiap mode,
- aturan retensi: **redaksi sebelum masuk prompt** = wajib `true`,
- sandbox: isolasi proses penuh, non-root, **tanpa** socket Docker, **tanpa** kredensial host,
- `approved_by` + `approved_at`,
- daftar larangan inti: bypass autentikasi, captcha, paywall, rate limit, DRM, certificate pinning, brute-force, pengambilan data backend.

Hasil: **PASS / FAIL / BLOCKED**, disimpan sebagai `reports/<target>/gate0-verdict.json`. FAIL = berhenti total. BLOCKED = minta keputusan manusia.

7.2 Release gate — bukti, bukan pernyataan

Aturan tertulisnya: **"Tidak ada PASS tanpa artefak baru. LLM statement bukan evidence."** Artefak wajib dan **umurnya maksimal 24 jam**:

Gate	Artefak wajib
Requirement	`requirements.md`
Build	`logs/build-*.log`
Tests	`logs/tests-*.log`
Review	`reports/review-*.md`
Security	`reports/security-gate.md` + `reports/security-verdict.json` (field `verdict` harus `PASS`)
Audit	`reports/assurance-audit.md`
Docs	`docs/README-*`, `reports/handover-*.md`

Penting: **artefak baru yang isinya FAIL tetap FAIL.** Keberadaan + kesegaran saja tidak pernah cukup. Ini menutup celah "jalankan tes, tes gagal, tetap bilang lulus".

7.3 Empat kelas risiko

Kelas	Contoh	Kontrol
LOW	dokumentasi, format, riset read-only	kontrol normal
MEDIUM	edit kode di workspace dev	branch git + tes + approval
HIGH	auth, pembayaran, rahasia, migrasi, CI/CD, infra	isolasi proses penuh + audit keamanan independen + approval manusia
CRITICAL	aksi destruktif produksi, rotasi kredensial, hapus data	**tanpa eksekusi otonom**, prosedur manusia, bukti rollback

7.4 Pemantau CI

`sdb-ci-watch.timer` memeriksa pipeline secara berkala (berikutnya terjadwal), melaporkan hasil ke kanal yang sama dengan laporan ekosistem. Fungsinya: mendeteksi "hijau kemarin, merah hari ini" tanpa menunggu Anda membuka GitHub.

---

8. Isi rumah: struktur folder dan artinya

Folder/berkas	Isi	Analogi
`README.md`	Pintu masuk, ringkasan kapabilitas	papan nama & brosur
`docs/` (35 berkas MD)	Blueprint, workflow, divisi, matriks model, tutorial	buku manual
`workflows/`	Definisi gate & pipeline (YAML)	SOP tertulis
`scripts/` (31 skrip)	Alat kerja: gate, workflow, laporan, bot, build	peralatan bengkel
`registry/`	Daftar skill & tool + status persetujuan	buku inventaris
`skills/` (40)	Skill SDB (cara kerja yang bisa dipanggil)	resep siap pakai
`agents/` (8 paket)	Definisi peran agen per divisi	kartu jabatan
`platform/packs/` (8)	Paket aturan per jenis pekerjaan	kit per jenis mobil
`evidence/` (8 target)	Bukti capture & otorisasi per target	berita acara
`runs/` (5)	Catatan eksekusi workflow	log harian bengkel
`reports/`	Laporan gate, audit, keamanan	sertifikat uji
`dashboard/`	Command Deck (port 8484) + state	ruang kendali
`sdb_platform/`	Control plane multi-tenant (port 8088)	loket layanan
`tests/` (36 berkas, 470 tes)	Uji otomatis	kalibrasi alat
`data/control.db`	DB control plane (tenant, job, tagihan)	buku kas
`~/.local/state/sdb/build_jobs.db`	Antrean build Telegram	papan order

Folder/berkas	Isi	Analogi
`~/sdb-builds/<id>-<slug>/`	Hasil build per job (sandbox)	ruang kerja terpisah

---

9. Perkakas: 31 skrip, 14 perintah `sdb`, 470 tes

9.1 Satu pintu masuk: `scripts/sdb.py`

Perintah	Fungsi
`sdb gate <target>`	Jalankan Gate #0 → PASS/FAIL/BLOCKED
`sdb init <target>`	Buat workspace target (evidence/normalized/...)
`sdb capture <target>`	Capture terkendali (wajib Gate #0 PASS)
`sdb normalize <target>`	Ubah artefak capture jadi JSON bersih
`sdb redact <file>`	Hapus rahasia dari artefak jaringan
`sdb lint <target>`	Evidence lint (metadata, confidence, redaksi)
`sdb workflow <target>`	Jalankan 16 tahap berbasis bukti
`sdb release <target>`	Release gate berbasis artefak
`sdb fidelity <a> <b>`	Bandingkan asli vs hasil bangun ulang
`sdb fidelity-report`	Laporan kemiripan 9 dimensi
`sdb scan <target>`	Pindai pola eksekusi/eksfiltrasi pihak ketiga
`sdb inventory`	Inventaris ekosistem
`sdb doctor`	Cek kapabilitas host + drift skill + pin dependency
`sdb install` / `uninstall`	Pasang/lepas skill & CLI (dry-run default)
`sdb list-skills`	Daftar/pasang skill ke profil

Perintah	Fungsi
`sdb registry-lint`	Validasi daftar skill/tool

9.2 Skrip pendukung penting

Skrip	Gunanya
`gate_check.py`	Mesin Gate #0
`release_check.py`	Mesin release gate (kesegaran 24 jam + verdict)
`evidence_lint.py`	Periksa mutu bukti
`ci_watch.py`	Pemantau pipeline
`secret_scan` (via make `secrets`)	Cari rahasia sebelum commit
`repo_hygiene.py`	Kebersihan repo (berkas nyasar, artefak besar)
`inventory_ecosystem.py`	Potret isi ekosistem
`workflow_run.py`	Runner 16 tahap
`fidelity_compare.py`	Ukur kemiripan hasil bangun ulang
`sdb_report.py`	Laporan ekosistem deterministik (dipakai bot 4 jam)
`sdb_jobs.py`	Antrean & status job build
`sdb_build_worker.py`	Pekerja build bertahap (systemd)
`sdb_tg_bot.py`	Bot Telegram (owner-only)
`md_to_pdf.py`, `build_tutorial.py`	Ubah dokumen MD → HTML + PDF

9.3 Target `make` yang sering dipakai

`make test` · `make lint` · `make check` · `make gate-init` · `make release-check` · `make secrets` · `make doctor` · `make inventory` · `make packs` · `make worker` · `make control-plane` · `make docs-tutorial` · `make hygiene` · `make drift`.

9.4 Uji otomatis: 470 tes

- 36 berkas tes, **470 tes** terkumpul (`pytest --collect-only`).

- Mencakup: gate, release, evidence lint, laporan, bot Telegram, antrean build, control plane (isolasi tenant, kuota, idempotensi), redaksi, dsb.
- `make test` adalah syarat sebelum commit — bukan formalitas.

---

10. Skill dan tool: 40 skill sendiri, 17 tool terpasang

10.1 Skill (cara kerja yang bisa dipanggil)

40 skill custom (terdaftar di `registry/skills.yaml`, semuanya berstatus `ADD`), dikelompokkan menurut sifat pekerjaan:

Kelompok	Isi (nama nyata di registry)
Rekonstruksi & bukti (D1)	`reconstruction-scope-gate`, `target-inventory`, `ui-ux-state-mapper`, `observed-network-contract`, `technology-profiler`, `evidence-normalizer`, `reconstruction-spec`, `reconstruction-fidelity`, `rea-reverse-engineering`, `replica-skill-pipeline`
Arsitektur (D3)	`architecture-decision-record`, `technology-selection`, `architecture-docs`, `architecture-compliance-audit`
Rekayasa (D4/D5)	`api-contract`, `feature-workflow`, `bug-workflow`, `chore-workflow`, `database-migration-safety`, `authz-authn-review`, `accessibility-audit`, `geo-audit`, `html-to-video-render`, `realtime-visual-engine`, `local-media-generation`, `model-hub-provenance`, `local-llm-serving`, `privilege-free-tooling`, `public-feed-reader`
Mutu, audit & rilis (D7/D9/D10)	`test-pyramid-and-risk`, `engineering-release-audit`, `ci-cd-release-gate`, `rollback-and-recovery`, `release-handover-docs`, `sdb-command-deck`, `local-experiment-tracking`
Produk / PLANORA (D0/D2)	`product-kit`, `product-status`, `product-change`, `product-execution`

> Catatan hitungan: **40** adalah skill milik SDB di registry. Hermes (host) juga punya skill bawaannya sendiri — mis. `third-party-tool-vetting`, `security-static-review`, `skill-inspector`, `plan`, `test-generator` — yang **tidak** masuk angka 40 dan tidak dihitung sebagai aset SDB.

17 tool pihak ketiga terdaftar dengan status eksplisit: **2 terpasang** (`trailofbits/skills`, `morluto/rea`), **1 dipakai** (`microsoft/playwright`), **3 ditolak**, **4 karantina** (MobSF, jadx, Apktool, ghidra), **1 ditunda** (BackstopJS), **6 belum terpasang**. Alasan penolakan dicatat, bukan sekadar "tidak jadi":

Ditolak	Status	Alasan
`obra/superpowers`	`REJECTED_BLUEPRINT`	pendekatannya sudah menjadi blueprint native SDB
`getsentry/skills`	`REJECTED_COVERED`	kapabilitasnya sudah tercakup skill SDB sendiri
`anthropics/skills`	`REJECTED_LICENSE`	lisensi tidak cocok untuk dipakai di sini

Aturan masuk tool baru (7 langkah, tidak boleh dilompati): `source_trust_check` → `pin_revision` → `hermes_native_scan` → `manual_source_review` → `sandbox_test` → `allowlist_tools` → `human_approval`.

10.2 Tool terpasang (versi dipin, bukan "latest")

Tool	Versi	Peran
git	2.43.0	jejak versi & bukti
node / npm	26.7.0 / 11.19.0	tooling frontend
docker	29.8.2	isolasi proses penuh (socket **tidak** diberikan ke pekerjaan untrusted)
uv	0.12.5	isolasi tool Python
pytest	9.1.1	runner tes
playwright	1.63.0	mesin capture utama (Chromium)
jq	1.7	olah bukti JSON
ruff	0.16.10	lint/format Python
semgrep	1.179.0	SAST cepat (read-only, metrik mati)
trivy	0.75.0	dependency/container/IaC/secret scan
syft	1.54.0	SBOM (SPDX/CycloneDX)

Tool	Versi	Peran
codeql	2.27.1	SAST mendalam — **hanya** untuk kode open source/riset, dilarang untuk kode proprietary & CI
llama.cpp server	0.5.0-dev (commit dipin)	LLM lokal, bind 127.0.0.1 saja
ffmpeg	build host	encode video bukti
Chromium headless	playwright 1.63.0	render visual realtime
rea-agents	6.8.0	reverse engineering artefak terkirim (JS/Electron/web)

Prinsip: **versi dipin + bukti pemasangan + persetujuan manusia**. Bukan `npm i` lalu berharap.

---

11. Ruang kendali (Command Deck) dan control plane

11.1 Command Deck — jendela pandang di PC ini

- Alamat: `http://127.0.0.1:8484` (layanan `sdb-command-deck`, aktif).
- Isi: papan status agen, monitor pekerjaan, tampilan "cast", panel intake (draf → Approve → Kirim), dan **PIN workspace** sebagai pintu masuk.
- Catatan: layanan ini bind ke `0.0.0.0` agar bisa dibuka dari perangkat lain di jaringan rumah. Artinya **bisa dijangkau dari LAN**; PIN adalah lapisan pertama, bukan satu-satunya. Kalau tidak perlu diakses dari perangkat lain, sebaiknya diubah ke `127.0.0.1`.

11.2 Control plane — loket layanan multi-tenant (tahap awal)

- Alamat: `http://127.0.0.1:8088` (layanan `sdb-control-plane`, aktif).
- Dibangun dengan **stdlib** `http.server`, sengaja tanpa framework: lebih sedikit permukaan serangan, lebih mudah diaudit.
- Endpoint: `/v1/tenants`, `/v1/jobs`, `/v1/usage`, `/v1/billing`, `/v1/invoices`, `/v1/ledger`, `/v1/credits/topup`, `/v1/admin/revenue`, `/v1/admin/invoices`, `/v1/worker/claim`, `/v1/jobs/<id>/artifacts`, `/v1/jobs/<id>/heartbeat`.
- Isi DB saat ini: **0 tenant, 0 job, 0 tagihan** — jadi ini "loket yang sudah berdiri tapi belum ada antrean". Jujur: belum dipakai untuk pelanggan nyata.

---

12. Bot Telegram SDB: tiga kanal operasional

Bot: `@super_dev_builders_bot`. Layanan `sdb-tg-bot` aktif. **Hanya chat Anda** yang dilayani; pesan dari chat lain diabaikan tanpa balasan (tidak ada "bot menolak" yang membocorkan keberadaan bot).

12.1 Kanal 1 — laporan otomatis tiap 4 jam

- Timer `sdb-report-4h.timer`: 00/04/08/12/16/20 WIB.
- Isi laporan: proyek + progres (berkas, baris, commit, perubahan belum di-commit), layanan hidup/mati, timer, Command Deck, `control.db`, kondisi host, antrean build, dan **delta sejak laporan sebelumnya**.
- **Deterministik tanpa LLM** → 0 biaya token, tidak bisa berhalusinasi. Snapshot disimpan di `dashboard/state/report_snapshot.json`.

12.2 Kanal 2 — tanya kapan saja

Perintah	Jawaban
`/status` `/health` `/revenue`	ringkas
`/eco`	kondisi ekosistem lengkap
`/projects`	daftar proyek + progres
`/ask <pertanyaan>`	jawaban atas pertanyaan bebas

`/ask` menyuntikkan laporan nyata sebagai **DATA** dan hanya diberi toolset `terminal, file, skills` (tanpa web/browser/delegasi). Tujuannya: jawaban ber-grounding, tidak melebar jadi aksi.

12.3 Kanal 3 — perintah membuat produk

Perintah	Arti
`/build <deskripsi>`	Buat spesifikasi + rencana + daftar pertanyaan (tanpa menulis kode dulu)
`/approve <id>`	Setujui spesifikasi → mulai build
`/job <id>`	Lihat status job
`/jobs`	Daftar job

Perintah	Arti
`/revise <id> <arahan>`	Revisi spesifikasi sebelum/sesudah build
`/answer <id> <jawaban>`	Jawab pertanyaan spesifikasi
`/reject <id>`	Tolak spesifikasi
`/retry <id>`	Lanjutkan job `paused` tanpa mengulang dari nol
`/cancel <id>`	Batalkan
`/help`	Daftar perintah

---

13. Alur membuat produk lewat Telegram — langkah demi langkah

13.1 Alurnya

```
/build "katalog villa + booking WhatsApp + admin"
   │
   ├─ 2–5 menit → SPEC.md (kebutuhan), PLAN.md (tahap), QUESTIONS.md (yang belum jel
   │              status: awaiting_approval  ← TIDAK ada kode ditulis di sini
   │
/approve 1
   │
   ├─ build bertahap: ronde 1, 2, 3 (maks. 3) + 1 ronde dokumentasi
   │  sandbox: ~/sdb-builds/001-<slug>/
   │  bukti: README.md + VERIFY.md (perintah + output nyata) + GAPS.md
   │
   └─ status: done  → notifikasi + ringkasan bukti
        (kalau belum tuntas) status: paused → /retry <id>
```

13.2 Kenapa dua tahap?

Karena kesalahan termahal adalah **membangun barang yang salah dengan rapi**. Tahap spesifikasi memaksa kebutuhan ditulis lebih dulu, dan Anda bisa menolak sebelum satu baris kode pun ditulis. Ini

penghemat waktu, bukan birokrasi.

13.3 Bukti nyata: job #1 "Villa Nusantara"

Yang diminta: katalog villa + tombol booking WhatsApp + panel admin (CRUD).

Aspek	Hasil terukur
Tumpukan	Next.js 15 (App Router) + SQLite
Tes unit	**44 lulus**
Build produksi	**compile sukses** (`npm run build`)
Smoke test	**55/55 PASS**
Data contoh	9 villa diseed, **8 tampil** di katalog
Dokumentasi	`README.md` , `VERIFY.md` (isinya perintah + output asli), `GAPS.md`
Lokasi	`~/sdb-builds/001-website-katalog-villa-dengan-fil/`
Status	`done` — dinilai dari **keberadaan** README + VERIFY, bukan dari klaim agen

Contoh isi `VERIFY.md` (ringkas): perintah build, perintah tes, jumlah lulus/gagal, dan potongan output apa adanya — termasuk catatan kalau ada yang belum selesai.

13.4 Batas yang dipasang pada pekerja build

- Satu job diproses **satu per satu** (job lain mengantre).
- Batas turn per ronde, batas anggaran, dan batas waktu dinding; proses dibunuh per process-group kalau melewati batas.
- Kerja dipaksa di dalam `~/sdb-builds/<id>-<slug>/` .
- Job dianggap selesai hanya jika README + VERIFY benar-benar ada.

---

14. Keamanan berlapis: apa yang dijaga dan bagaimana

Lapisan	Yang dijaga	Caranya
Identitas	Hanya pemilik yang bisa memerintah	Cek `chat_id` terhadap `SDB_TG_CHAT_ID` ; chat lain diabaikan diam-diam

Lapisan	Yang dijaga	Caranya
Rahasia	Token bot tidak bocor ke kode hasil build	Token dibaca dari `EnvironmentFile` (mode 600), **dihapus** dari env proses anak
Izin	Tidak membangun/membedah tanpa hak	Gate #0 + `authorization.yaml` + verdict JSON
Isolasi	Pekerjaan tidak menyentuh hal lain	CWD dikunci ke sandbox, larangan menyentuh `~/.hermes`, `~/.config`, repo lain
Mutu	Tidak ada "selesai palsu"	Release gate: artefak wajib, umur ≤ 24 jam, verdict harus PASS
Rantai pasok	Tool/skill tidak membawa malware	7 langkah vetting + versi dipin + tinjauan manusia
Audit	Semua langkah bisa ditelusuri	`events` per job, log systemd, `reports/*.json`, commit git

Yang perlu Anda tahu secara jujur: isolasi untuk pekerja build saat ini bersifat **tingkat instruksi + direktori kerja**, belum container OS terpisah. Artinya agen build berjalan sebagai user Anda. Mitigasinya: approval dua tahap, hanya pemilik, batas waktu, dan jejak audit. Kalau nanti dipakai untuk pekerjaan dari pihak luar, langkah berikutnya adalah menjalankan worker di container/VM terpisah.

---

15. Cara pakai harian — cheat sheet

15.1 Di Telegram (paling praktis)

```
/eco                     kondisi ekosistem sekarang
/projects                progres proyek
/ask kenapa build tadi lama?   tanya bebas
/build website katalog kue + admin
/approve 2
/job 2                   cek status
/retry 2                 lanjutkan kalau paused
```

15.2 Di terminal PC ini

```
cd ~/super-dev-builders

make test                 # 470 tes, wajib hijau sebelum commit
make check                # lint + tes + hygiene
make doctor               # kapabilitas host + drift
make inventory             # potret ekosistem
make secrets               # cari rahasia sebelum commit

.venv/bin/python scripts/sdb_report.py --text     # laporan yang sama dengan bot
.venv/bin/python scripts/sdb_jobs.py stats        # antrean build
./scripts/sdb.py gate <target>                    # gate izin
./scripts/sdb.py workflow <target>                # 16 tahap
./scripts/sdb.py release <target>                 # release gate
```

15.3 Melihat hasil build

```
cd ~/sdb-builds/001-*/ && cat VERIFY.md && npm run dev
```

15.4 Kalau laporan Telegram berhenti

```
systemctl --user status sdb-report-4h.timer sdb-tg-bot
systemctl --user list-timers 'sdb*'
journalctl --user -u sdb-report-4h -n 50 --no-pager
```

---

16. Batasan jujur: apa yang belum ada

Bagian ini sengaja tidak dibungkus rapi. Kalau sebuah dokumen tidak punya bagian ini, dokumen itu iklan, bukan laporan.

Belum ada / terbatas	Dampak	Rencana
Control plane belum dipakai (0 tenant)	Belum ada pelanggan nyata	Diisi setelah ada kebutuhan multi-tenant

Belum ada / terbatas	Dampak	Rencana
Isolasi build belum container	Agen build berjalan sebagai user	Pindah ke container/VM saat dipakai pihak luar
Model routing bergantung ketersediaan provider	Bisa turun kualitas saat provider down	Ada jalur cadangan (mis. GPT-5.6 Sol untuk security)
CodeQL dilarang untuk kode proprietary	Analisis mendalam terbatas pada kode berlisensi	Pakai semgrep/trivy + review manual
Deck bind 0.0.0.0	Terjangkau dari LAN	Ubah ke 127.0.0.1 bila tidak perlu remote
Belum ada auto-deploy produksi	Rilis tetap manual (memang disengaja)	Jangan diotomatiskan untuk HIGH/CRITICAL
Job build contoh baru 1 (Villa)	Bukti pipeline, belum bukti skala	Tambah job nyata berikutnya

---

17. Glosarium bahasa awam

Istilah	Arti sederhana
ADR	Catatan keputusan: apa yang dipilih, kenapa, apa alternatifnya
Artefak	Barang bukti hasil kerja: log, laporan, JSON, diff
Gate	Pintu pemeriksaan; tidak lolos = tidak boleh lanjut
Deterministik	Hasilnya sama setiap dijalankan; tidak mengarang
Sandbox	Ruang kerja terpisah supaya tidak merusak hal lain
SBOM	Daftar bahan (library) yang dipakai sebuah aplikasi
SAST	Pemeriksa kode otomatis untuk pola berbahaya
Tenant	Satu pelanggan/pengguna terpisah dalam satu sistem
Idempotensi	Menekan tombol dua kali tidak membuat dua data

Istilah	Arti sederhana
Rollback	Jalan pulang ke kondisi sebelum perubahan
Smoke test	Uji cepat "apakah nyala dan fungsi dasar jalan"
Turn	Satu giliran agen memanggil alat lalu menjawab
Fidelity	Seberapa mirip hasil bangun ulang dengan aslinya
Evidence	Bukti yang bisa diperiksa mesin, bukan cerita

---

18. Peta dokumen rujukan

Dokumen	Isi
`docs/BLUEPRINT_V2.2.md`	Cetak biru lengkap ekosistem
`docs/WORKFLOW_GATES.md`	16 tahap + isi gate + kelas risiko
`docs/DIVISIONS.md`	12 divisi dan pembagian peran
`docs/MODEL_ROUTING_MATRIX.md`	Model mana untuk tahap mana
`docs/TELEGRAM_SDB.md`	Detail teknis bot Telegram
`docs/TUTORIAL_PENGGUNAAN.md`	Tutorial pemakaian (HTML/PDF tersedia)
`docs/TUTORIAL_PRIBADI.md`	Panduan pribadi: deck, shortcut, bot
`docs/PETA_STRUKTUR.md`	Peta struktur repo
`README.md`	Ringkasan kapabilitas
`registry/skills.yaml`, `registry/tools.yaml`	Inventaris skill & tool
`workflows/*.yaml`	Definisi mesin dari gate

---

19. Lampiran A — angka terverifikasi dan cara mengeceknya

Semua di bawah diambil **4 November 2026** dari mesin ini, bukan dari ingatan.

Angka	Nilai	Perintah pengecekan
Commit	130 (terakhir `852c0ab`, 2026-11-03)	`git rev-list --count HEAD`
Berkas Python	103	`find . -name '*.py' -not -path './skills/*' -not -path './.venv/*' | wc -l`
Baris Python	18.748	`find ... -print0 | xargs -0 cat | wc -l`
Berkas tes	36	`ls tests/test_*.py | wc -l`
Jumlah tes	**470**	`pytest --collect-only -q | awk -F': ' '/^tests\//{s+=$2} END{print s}'`
Dokumen MD di `docs/`	35	`ls docs/*.md | wc -l`
Skrip di `scripts/`	31	`ls scripts/*.py | wc -l`
Skill custom	40	`registry/skills.yaml` → `custom`
Tool pihak ketiga	17 (3 ditolak)	`registry/skills.yaml` → `third_party`
Tool host terpasang	17	`registry/tools.yaml` → `present`
Paket divisi	8	`ls platform/packs`
Paket agen	8	`ls agents`
Layanan aktif	3 (`sdb-tg-bot`, `sdb-build-worker`, `sdb-command-deck`)	`systemctl --user list-units 'sdb*'`

Angka	Nilai	Perintah pengecekan
Timer	3 (laporan 4 jam, CI watch, deck health)	`systemctl --user list-timers 'sdb*'`
Port	8484 (deck), 8088 (control plane)	`ss -lptn`
Job build	1 selesai (Villa Nusantara), 22 event	`scripts/sdb_jobs.py stats`

Hasil uji terakhir

```
pytest: 470 tes terkumpul; suite penuh hijau (make test)
build job #1: 44 tes unit lulus · build Next.js compile · smoke 55/55 PASS
gate/release/evidence lint: hijau
```

---

20. Lampiran B — riwayat dokumen

Versi	Tanggal	Perubahan
1.0	2026-11-04	Dokumen paket update menyeluruh pertama: 16 tahap, 12 divisi, gate, perkakas, deck, control plane, bot Telegram 3 kanal, bukti job #1, batasan, glosarium, lampiran verifikasi

Cara memperbarui dokumen ini: jalankan perintah di Lampiran A, bandingkan angkanya, lalu ubah angka + tanggal. Jangan menambah klaim baru tanpa artefak pendukung.

Dokumen ini bagian dari SDB — Divisi Ekosistem. Semua klaim memiliki jejak perintah yang bisa dijalankan ulang. Kalau ada yang tidak cocok dengan kenyataan, yang benar adalah kenyataan; dokumen ini yang harus diperbaiki.
